

Final hackathon idea

Flight Recorder

A real-time audit trail for what your coding agent actually did — and why

Complete project specification · v1.0 · 4-day build

1. WHAT — Product definition

Flight Recorder is a local, real-time black box for Claude Code sessions. It captures every consequential action an agent takes (shell commands, file edits, network calls, account/credential operations) *at the moment it happens*, binds each action to the reasoning that preceded it, and preserves everything in a local, append-only, greppable store with a live timeline viewer — so that "why did the agent do that?" is answerable forever, not just until the transcript compacts or auto-deletes.

The one-sentence demo: Ask *"why did the agent add this environment variable?" three weeks later, click the action, and read the exact reasoning that produced it.*

What it is NOT (scope discipline)

- Not an observability platform (Honeycomb Agent Timeline) — no cloud, no OTel pipeline, no enterprise dashboards.
- Not a policy engine — it records; it does not block. (Blocking is the sibling project. One risk-gate stretch feature is allowed at most.)
- Not a transcript summarizer (Grepthy) — we capture at action time via hooks, not by distilling expired chat logs.
- Not a memory/knowledge-base system (Decispher) — we are evidence, not context.

Core feature set (MVP — must all work by Day 3 evening)

#	Feature	Description
F1	Action capture	Every tool call Claude Code makes (Bash, Edit, Write, Read, WebFetch, MCP tools) recorded via hooks with full arguments and results
F2	Reasoning capture	The assistant messages/thinking immediately preceding each action, extracted from the live transcript at hook time — before compaction or deletion can destroy it

#	Feature	Description
F3	Context stamp	Timestamp, session ID, cwd, git branch + HEAD + dirty status at the moment of each action
F4	Risk classification	Every action auto-tagged: <code>info</code> / <code>write</code> / <code>exec</code> / <code>network</code> / <code>sensitive</code> (credentials, auth, account ops, destructive commands)
F5	Append-only local store	SQLite (queryable) + JSONL mirror (greppable with <code>cat/grep/jq</code> — explicit ask from the HN thread)
F6	Live timeline viewer	Web UI: watch actions stream in live, filter, search, drill into any action's "why"
F7	Compaction shield	PreCompact hook snapshots the transcript before Claude Code compacts it — reasoning survives even when context doesn't

Stretch features (only after MVP is demo-solid)

- S1: Incident report export — select a time range → one markdown report (what happened, in order, with reasoning excerpts)
- S2: Multi-session view — all recorded sessions in one list, cross-session search
- S3: File snapshot diffs — before/after content capture for edits beyond what `tool_input` provides
- S4: "Sensitive action" desktop notification — recorder pings you when a `sensitive`-tier action fires

2. WHY — Problem, evidence, positioning

The problem

Coding agents take real actions with real side effects: they create accounts, modify configs, run migrations, touch credentials. The *actions* are sometimes recoverable (git, shell history). The *reasoning* is not — it lives only in the session transcript, which gets compacted mid-session (context pressure) and auto-deleted after 30 days. Once that happens, an unexplained change in your infrastructure has no explanation, forever.

The evidence (recent, independent, repeated)

1. **The Grepathy Show HN (July 15, 2026):** "Claude made a decision nobody approved" — Claude Code silently created guest accounts; the reasoning trail was lost when transcripts auto-deleted. Commenters explicitly said the deeper gap — an audit trail *baked into the tooling in real time*, not a post-hoc bolt-on — remains unsolved. That comment is our product brief.

2. **Real-world horror stories in circulation:** "Claude Code deleted 92 images without asking" and similar posts; teams running agents on cron asking "what did it do at 3am?"
3. **Enterprise pull:** consultancies report financial-services and healthcare clients demanding "a complete audit trail of everything an AI agent does — not just the conversation transcript" via SDK hooks. We're productizing what they hand-roll.

Competitive landscape (and our exact wedge)

Existing thing	What it is	Why we're different
Grepathy	Post-hoc transcript distillation to markdown	We capture at action time via hooks; nothing to lose to expiry
Honeycomb Agent Timeline	Enterprise cloud observability for production agents	We're local-first, zero-config, for the dev's own machine; audit logs of your codebase never leave your disk
AIR Blackbox	Records/replays LLM calls for framework-built agents (LangChain etc.)	We target Claude Code sessions — the agent people actually run daily — via native hooks, no code changes
Decispher	Knowledge base of decisions to feed context back to agents	We're an evidence log, not a memory system; different job
AgentsRoom	Visual monitor of hook firings across sessions	Monitoring ≠ audit: we bind <i>reasoning</i> to actions and preserve it durably
DIY hook scripts (37-line JSONL loggers)	Log tool names to a file	They capture <i>what</i> ; we capture <i>what + why + git state + risk</i> , with a viewer

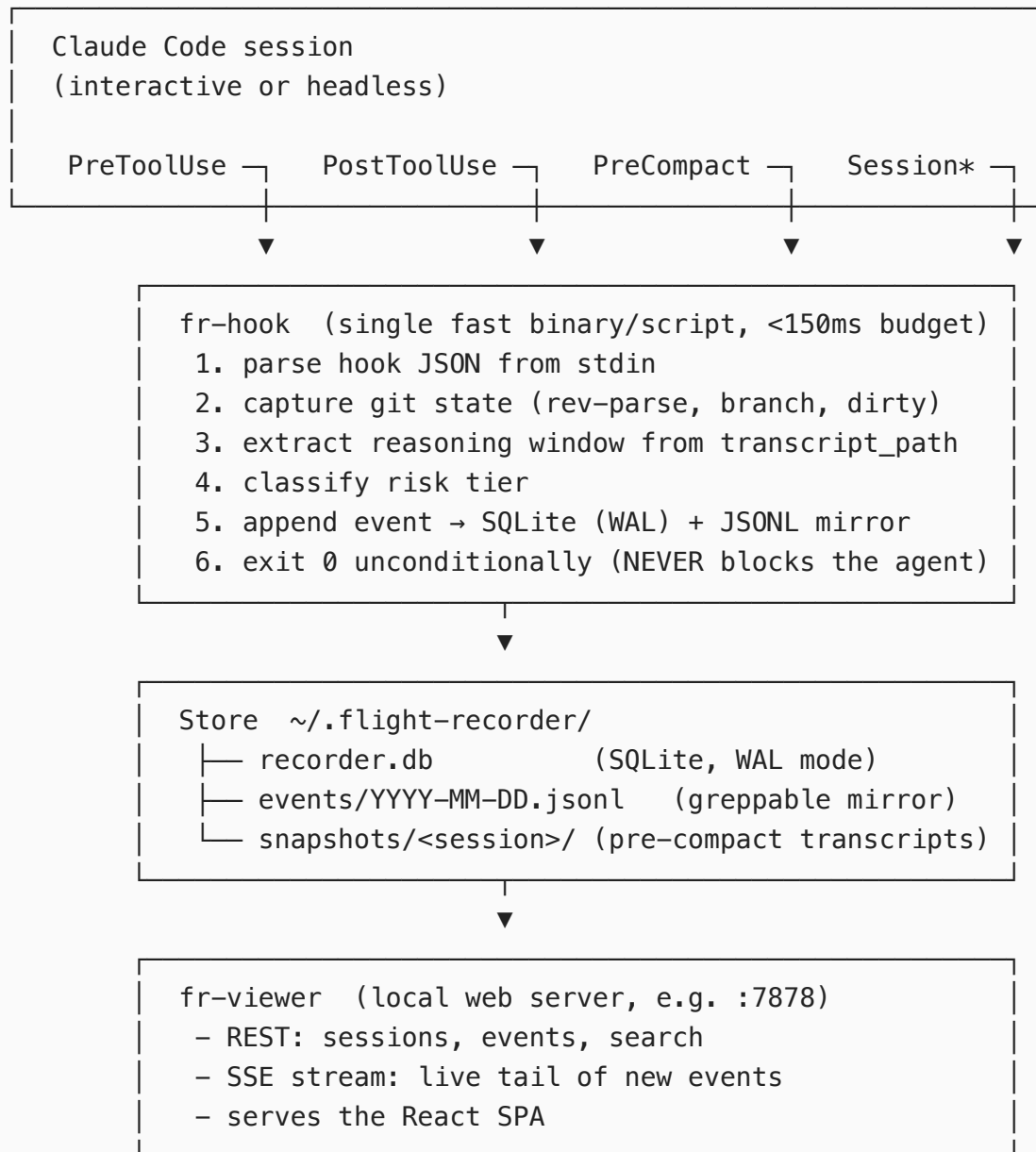
The wedge in one line: everyone else records *what happened*; nobody durably records *why* — and "why" is the thing the transcript deletion destroys.

Why now

- Claude Code's hooks API is mature and official: PreToolUse, PostToolUse, UserPromptSubmit, Stop, SubagentStop, SessionStart/End, PreCompact — with documented JSONL stdin payloads including `session_id`, `transcript_path`, `cwd`, `tool_name`, `tool_input`, `tool_response`. The docs literally list "log and audit every tool call for compliance" as an intended use. The platform is inviting this product; nobody has shipped the polished version.
 - Agent autonomy is rising (headless mode, CI, cron, long-running sessions) — exactly the settings where nobody is watching and the audit trail matters most.
-

3. HOW — Architecture & system design

3.1 High-level architecture



Key design decision — no daemon. The hook writes directly to SQLite in WAL mode (safe for concurrent writers + one reader). This removes an entire failure class (daemon not running = silent data loss) and makes install one command. The viewer is just a reader.

3.2 Component responsibilities

fr-hook (Python, single file, stdlib + sqlite3 only)

- Registered in `.claude/settings.json` (project) or `~/.claude/settings.json` (global) for: `PreToolUse` (matcher: *), `PostToolUse` (matcher: *), `PreCompact`, `SessionStart`, `SessionEnd`, `Stop`.
- Reads the event JSON from stdin. Never trusts fields to exist; degrades gracefully.

- **Performance budget: p95 < 150ms** (PostToolUse fires on every tool call; slow hooks = laggy agent). Git commands run with a 100ms timeout; reasoning extraction reads only the tail of the transcript file (last N KB, not the whole file).
- **Prime directive: exit 0 no matter what.** A recorder that can break the pilot's controls is worse than no recorder. All exceptions swallowed to a debug log.

Reasoning extraction (the crown jewel — F2)

- The hook payload includes `transcript_path` → the session's JSONL transcript on disk.
- On PreToolUse: seek to end of file, read backward up to 64KB, parse JSONL lines, collect assistant text/thinking blocks since the last tool event → this is the "reasoning window" for the incoming action. Store verbatim (truncated at 8KB per event, full text retrievable from snapshot).
- On PreCompact: copy the entire transcript file to `snapshots/` before Claude Code compacts it. This is the moment reasoning would otherwise die; we are the shield. Tag the snapshot with session + timestamp.
- Fallback if transcript unreadable: record the action anyway with `reasoning: null` and a `capture_gap: true` flag — honesty over pretending.

Risk classifier (deterministic, table-driven — no LLM in the hot path)

Tier	Color	Triggers (examples)
info	gray	Read, Glob, Grep, LS
write	blue	Edit, Write, NotebookEdit
exec	amber	Bash (default)
network	purple	WebFetch, curl/wget/ssh/scp in Bash commands, MCP network tools
sensitive	red	Regex hits in command/args: <code>rm -rf</code> , <code>sudo</code> , <code>chmod 7</code> , <code>.env</code> , <code>secret</code> , <code>token</code> , <code>credential</code> , <code>password</code> , <code>useradd</code> , <code>adduser</code> , account-creation API patterns, <code>DROP TABLE</code> , <code>git push --force</code> , keychain paths

Rules live in one `risk_rules.yaml` — demoable ("add your own rule live") and honest about being heuristic.

fr-viewer (FastAPI + SSE, serving a Vite/React/Tailwind SPA)

- Endpoints: `GET /api/sessions`, `GET /api/sessions/{id}/events?filter=...`, `GET /api/events/{id}` (full detail incl. reasoning), `GET /api/search?q=...`, `GET /api/stream` (SSE tail), `GET /api/report?from=&to=` (stretch S1).
- Search grammar (tiny, hand-rolled): free text over args/reasoning + qualifiers
`tool:bash`, `risk:sensitive`, `file:auth`, `session:<prefix>`. SQLite FTS5 index

over arguments_text + reasoning_text.

CLI (fr)

- `fr init` — writes hook config into `.claude/settings.json` (merge, don't clobber), creates store, prints "recording •".
- `fr status` — hooks installed? events flowing? last event age.
- `fr ui` — starts viewer, opens browser.
- `fr grep <pattern>` — thin wrapper over the JSONL mirror for terminal purists (it's literally grep with the right paths — the point is discoverability).

3.3 Data model (SQLite DDL, abbreviated)

```
CREATE TABLE sessions (
  id TEXT PRIMARY KEY,           -- claude session_id
  started_at INTEGER, ended_at INTEGER,
  cwd TEXT, git_repo TEXT, source TEXT -- interactive | headless |
  resumed
);

CREATE TABLE events (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id TEXT REFERENCES sessions(id),
  ts INTEGER NOT NULL,           -- ms epoch
  phase TEXT NOT NULL,           -- pre | post | compact | session
  tool TEXT,                     -- Bash, Edit, Write, WebFetch, mcp_*,
  ...
  arguments_json TEXT,           -- full tool_input
  result_json TEXT,              -- tool_response (post), truncated 16KB
  exit_ok INTEGER,               -- derived success flag
  risk TEXT NOT NULL DEFAULT 'info',
  risk_reasons TEXT,             -- which rules fired
  reasoning_text TEXT,           -- the captured "why" window
  capture_gap INTEGER DEFAULT 0,
  git_branch TEXT, git_head TEXT, git_dirty INTEGER,
  files_touched TEXT             -- JSON array, derived from Edit/Write
  args
);

CREATE VIRTUAL TABLE events_fts USING fts5(
  arguments_text, reasoning_text, content='events', content_rowid='id'
);
```

JSONL mirror: one object per line = the same event row, written in the same hook invocation. Crash-safe (append-only), valid forever, `jq`-friendly.

3.4 Event flow (one action, end to end)

1. Claude decides to run `Bash: npm create-user-account ...`
2. **PreToolUse** fires → `fr-hook` reads stdin JSON → tails transcript → reasoning window = "I need a service account for the staging deploy, I'll create one non-interactively..." → git state captured → risk classifier: `sensitive` (account-creation pattern) → row inserted (`phase=pre`) → JSONL appended → exit 0. (Claude proceeds; we never block.)
3. Command runs.
4. **PostToolUse** fires → `fr-hook` records result, exit status, updates the pre-row's pairing (matched by session + tool + monotonic ordering) → SSE pushes to any open viewer.
5. Viewer timeline shows a red-tier entry sliding in, live. Click → drawer opens: command, output, git context, and the reasoning paragraph. The "why" is one click away, forever.

3.5 Failure modes & handling

Failure	Handling
Hook crashes / bad JSON	try/except everything → debug log → exit 0. Agent never affected.
Transcript unreadable / moved	Event recorded with <code>capture_gap=1</code> ; UI shows a subtle "reasoning unavailable" state, never fake text
Two concurrent sessions	WAL handles concurrent writers; events keyed by <code>session_id</code> ; viewer separates them
Huge outputs (build logs)	result truncated at 16KB with <code>truncated: true</code> marker
Store corruption	JSONL mirror is the recovery source; <code>fr rebuild (stretch)</code> re-ingests
Hook adds latency	150ms budget, timeouts on subprocesses, tail-read only; measured in Day 1 acceptance test
User privacy	Everything local. No network calls in the hook. Ever. This is a headline feature, not an afterthought.

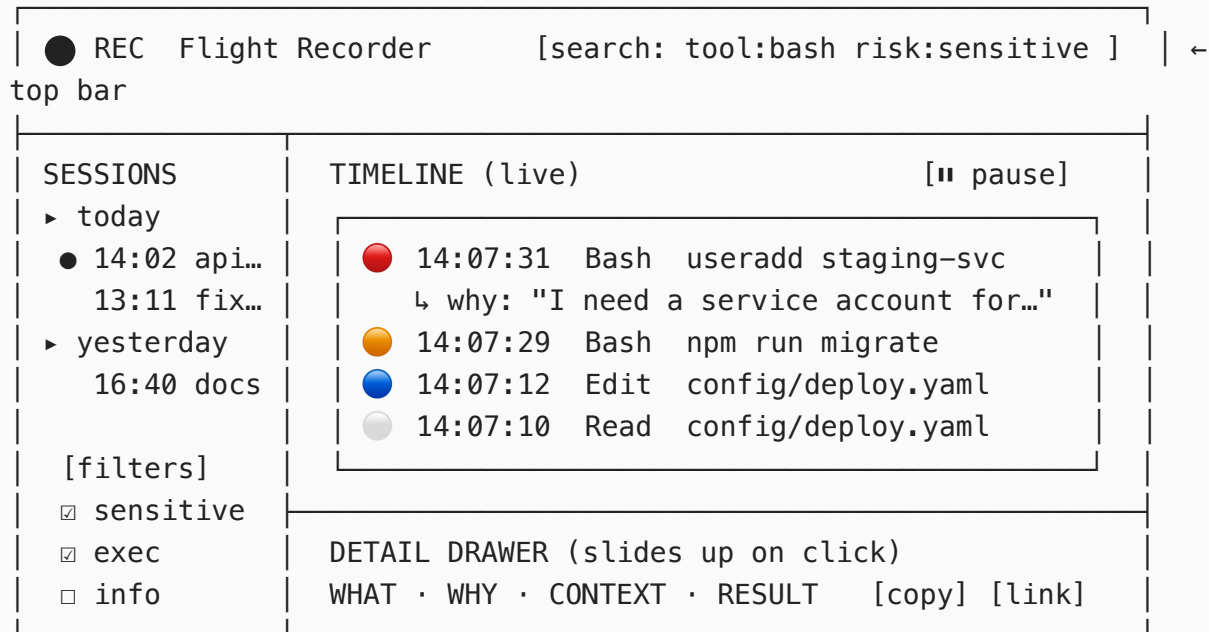
4. UI/UX — The viewer in detail

4.1 Design principles

1. **Aviation black-box aesthetic, but readable.** Dark theme, monospace accents, a persistent ● REC indicator when events are flowing. Serious tool, light theatrical touch.
2. **The timeline is the product.** One vertical stream, newest at top in live mode; risk color is the primary visual channel; everything else is one click deep.
3. **"Why" is never more than one click away.** Every action row → drawer with reasoning. This interaction IS the demo.

4. **Zero-state teaches.** Empty timeline shows the three commands to get recording (`fr init` , start a Claude session, watch).

4.2 Layout (single-page, three zones)



Timeline row anatomy: risk dot · time · tool badge · one-line summary (command or file path) · inline first-line-of-reasoning in muted italics ("↳ why: ...") — the row alone answers 80% of questions.

Detail drawer (the money view), four tabs in one panel:

- **WHAT** — tool, full arguments (syntax-highlighted), files touched
- **WHY** — the captured reasoning window, verbatim, with a timestamp and a "captured live before compaction" badge. If `capture_gap`: an honest amber note instead.
- **CONTEXT** — cwd, git branch, HEAD (short), dirty flag, session link
- **RESULT** — output/exit status, truncation marker if any

Live behavior: SSE feed; new rows slide in at top; auto-scroll pauses the moment the user scrolls (never fight the user); a `⏸` paused – 3 new pill returns them to live.

Search UX: one box, qualifier chips appear as you type (`risk:` , `tool:` , `file:` , `session:`); results replace the timeline; hitting Esc returns to live. Every event has a permalink (`/e/1234`) — "send your teammate the exact action."

Session sidebar: grouped by day; live session marked ●; clicking scopes the timeline; "All sessions" scope for cross-session search (stretch S2 makes this first-class).

4.3 Micro-interactions worth the time

- Risk dot pulses once on arrival for `sensitive` events.

- j/k keyboard navigation through rows, Enter opens drawer, / focuses search — devs live on keyboards, judges notice polish.
- Copy buttons produce clean markdown (feeds the S1 incident report story).

4.4 What we deliberately do NOT build

No auth/login (local tool), no settings UI (YAML is fine), no charts/analytics dashboards (audit ≠ analytics), no light theme (one theme, done well), no mobile layout.

5. Demo script (3 minutes, rehearsed Day 4)

1. **Cold open (30s):** "Last week on HN: 'Claude made a decision nobody approved' — guest accounts created, reasoning gone, transcript deleted. This is the black box that makes that impossible." Show `fr init` → ● REC .
 2. **Live capture (60s):** In a real repo, ask Claude Code: *"set up a staging config and a service account for deploys."* Viewer on the projector: rows stream in, the account-creation command lands **red**. Click it → WHY tab → read the agent's actual reasoning aloud.
 3. **The time-travel punch (45s):** Switch to a session recorded days ago (pre-seeded), whose transcript has been compacted/deleted. Ask the audience's question — "why does this env var exist?" — search `file:deploy.yaml risk:write`, open the event, reasoning intact. "The transcript is gone. The why is not."
 4. **The grep flex (15s):** Terminal: `fr grep 'useradd'` → JSONL lines. "And for the people who don't want a UI — it's just files. It's yours."
 5. **Close (30s):** Roadmap slide (policy gating sibling, team mode, incident reports). Backup video queued the entire time.
-

6. Risks & cut lines

Risk	Likelihood	Mitigation / cut line
Transcript format differs from expectation / changes	Medium	Day 1 spike task #1 is <i>empirically</i> dumping real hook payloads + transcript lines before writing any parser. Parser is defensive by design.
Reasoning window extraction is noisy	Medium	Verbatim > clever. Show the raw window; no summarization. If pairing reasoning→action is ambiguous, attach the whole inter-action window.
Hook latency annoys during dev	Low	Budget + measurement harness Day 1; worst case, move reasoning extraction to PostToolUse only.

Risk	Likelihood	Mitigation / cut line
UI polish eats Day 3	High	Drawer + timeline are the only two components that must be beautiful. Sidebar/search can be plain.
PreCompact rarely fires in short demo sessions	Certain	Pre-seed a long recorded session for the time-travel demo; PreCompact shield shown via the snapshot file's existence.
Global cut line	—	If behind by Day 3 noon: cut search UI (keep <code>fr grep</code>), cut session sidebar (single-session view), keep timeline + drawer + live SSE. Those three ARE the product.

7. WHEN — 4-day timeline

Assumes 2–3 people: **A = capture/backend**, **B = viewer/frontend**, **C (if exists) = CLI, demo content, docs**; otherwise A/B absorb.

Day 1 — Truth on the ground + skeleton (*goal: real events in SQLite by tonight*)

- ☐ **A (morning):** Spike — register throwaway hooks for every event type; dump raw stdin JSON + transcript samples from a real session into `docs/payloads/`. *No parser written before payloads observed.*
- ☐ **A (afternoon):** `fr-hook v1`: parse → git state → SQLite (WAL) insert → JSONL append → exit-0 armor. Latency harness (assert p95 < 150ms over 200 synthetic invocations).
- ☐ **B (morning):** Repo scaffolding: Python package + Vite/React/Tailwind app; SQLite schema + FTS migration; fake-event generator (lets frontend develop without live sessions).
- ☐ **B (afternoon):** FastAPI viewer skeleton: `/api/sessions`, `/api/events`, SSE stream wired to fake generator.
- ☐ **C:** `fr init` (settings.json merge logic — careful, don't clobber user hooks), `fr status`. README skeleton.
 - **Exit criteria:** run a real Claude Code task → `sqlite3 recorder.db 'select tool,risk from events'` shows it.

Day 2 — The crown jewels (*goal: reasoning + risk, end to end, real data*)

- ☐ **A:** Reasoning extraction (transcript tail-read, window pairing, 8KB truncation, `capture_gap` honesty path). PreCompact snapshot shield. Pre/Post event pairing.
- ☐ **A:** Risk classifier + `risk_rules.yaml`; unit tests fed with Day-1 captured payloads.

- ☐ **B:** Timeline component for real: row anatomy, risk colors, live SSE slide-in, pause-on-scroll. Detail drawer v1 (WHAT/WHY/CONTEXT/RESULT).
- ☐ **C:** Seed the "old session" dataset for the time-travel demo (run several real varied sessions today so they age + one long session that triggers compaction). Start demo video shot list.
 - **Exit criteria:** click any live action → read its actual reasoning in the drawer.

Day 3 — Search, polish, MVP freeze (*goal: demo-able by 6pm, feature freeze after*)

- ☐ **A:** FTS search endpoint + qualifier parsing; permalinks; `fr grep`.
- ☐ **B:** Search UX + filter chips; session sidebar; empty states; keyboard nav; ● REC indicator; sensitive-event pulse. Visual pass on the two hero components only.
- ☐ **C:** `fr ui` open-browser flow; install docs tested on a clean machine (this always breaks — do it today, not Day 4).
- ☐ **All, 6pm:** MVP freeze. Bug list triage: demo-path bugs only from here.
- ☐ **Evening (only if green):** stretch S1 incident report OR S2 multi-session — whichever is closer.
 - **Exit criteria:** full demo script executed once, ugly but complete.

Day 4 — Rehearse, record, harden (*goal: the demo cannot fail*)

- ☐ **Morning:** Full dress rehearsal ×3 with the projector setup. Fix only what breaks the script.
- ☐ **Midday:** Record backup demo video (screen + voiceover, the full 3-minute script). Non-negotiable.
- ☐ **Afternoon:** README (install, architecture diagram, honest limitations), demo seed data double-checked, submission form fields drafted, slides (≤5: problem/evidence, wedge table, architecture, roadmap, ask).
- ☐ **Buffer (2h minimum):** untouched. It will be consumed. That's what it's for.
 - **Exit criteria:** any team member can run the demo alone, and the video exists.

Hackathon-fit note

If this is being submitted to a track that scores usage of a specific build stack (per the earlier Build Week briefing): the event schema is deliberately agent-agnostic (`tool` , `arguments_json` , `phase`), so an adapter for another CLI agent's hook/log mechanism is a bounded Day-3-evening stretch — and "we built the entire recorder *with* the sponsor's agent, recording itself" is a genuinely great story: **the first session in the demo database is the session that built the product**. Decide this before Day 1; it changes nothing architecturally.

Ship the timeline. Ship the drawer. Everything else is negotiable.